

CS 660: Data Mining For Decision Making

Lecture 13 (Week 6)

Varun Dutt

School of Computing and Electrical Engineering
School of Humanities and Social Sciences
Indian Institute of Technology Mandi, India



Scaling the Heights

Neural Network: Perceptron or Neurode

- Connectionist Approach

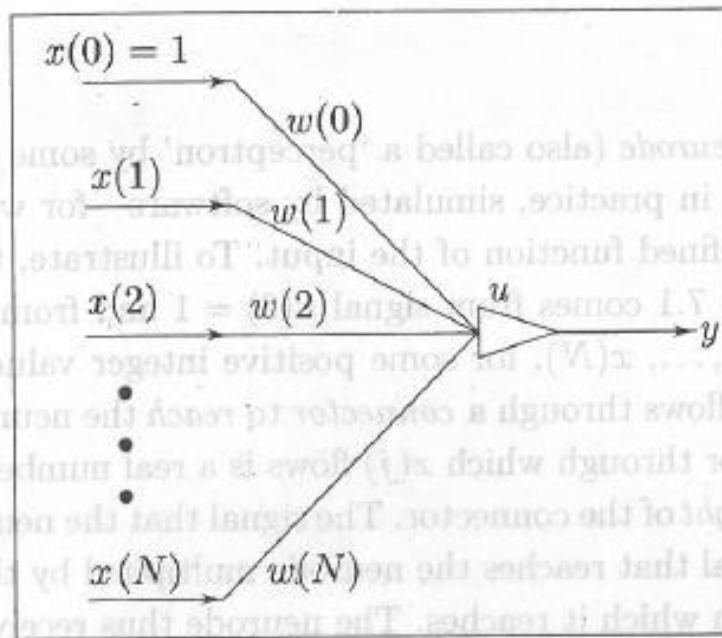


FIGURE 7.1

Real-valued signals $x(0)$ to $x(N)$ flow into neurode u , which outputs y . The weight of the connector through which $x(j)$ flows to u is $w(j)$, for $0 \leq j \leq N$. Overall, input I to u is equal to $\sum_{j=0}^N w(j)x(j)$. We can define a function so that output y is that function of input I .

Theoretical Equations

Let us define a *neurode* (also called a 'perceptron' by some writers) to be a processor—often, in practice, simulated by software—for which the output is a previously defined function of the input. To illustrate, the input to the neurode u of Fig. 7.1 comes from signal $x(0) = 1$ and from the real-valued signals $x(1), x(2), \dots, x(N)$, for some positive integer value of N . For $0 \leq j \leq N$, each $x(j)$ flows through a *connector* to reach the neurode. Associated with the connector through which $x(j)$ flows is a real number $w(j)$, which is said to be the *weight* of the connector. The signal that the neurode receives is, however, the signal that reaches the neurode multiplied by the weight of the connector through which it reaches. The neurode thus receives the *weighted signal* $w(j)x(j)$. Having $w(j) = 0$ is like having no connector through which $x(j)$ can reach the neurode. Overall, the input I to the neurode is then the sum of the weighted signals it receives, that is,

$$\begin{aligned} I &= \sum_{j=0}^N w(j)x(j) \\ &= w(0) + \sum_{j=1}^N w(j)x(j) \quad (\text{because } x(0) = 1) \end{aligned}$$

As we shall soon see, having $x(0)$ is a design choice. If we have $x(0)$, then fixing it to 1 is a convention. Even if all of $x(1)$ to $x(N)$ become zero, the neurode still receives the non-zero input of $w(0)$, which is said to be the *bias* received by the neurode. Having $w(0) = 0$ means having no $x(0)$. If there is no $x(0)$, the neurode's input comes from $x(1)$ to $x(N)$.

The output y of the neurode can be defined to be some function of the input I , for instance,

$$y = \begin{cases} 0 & \text{if } I < t \\ 1 & \text{otherwise} \end{cases}$$

$\Delta w(j) = \text{Beta} * (z - y) * x(j)$, where Beta is the learning rate and z is the desired output

Two kinds of updating: Epoch (after all training patterns) and Case (after each training pattern)

AND GATE via Neural Network

Training pattern	Attributes corresponding to		Class	Desired output z of neuron
	$x(1)$	$x(2)$		
1	1	1	YES-AND	1
2	0	1	NO-AND	0
3	1	0	NO-AND	0
4	0	0	NO-AND	0

Training set for a neuron to model an AND gate. The desired output z of the neuron is different for the two classes, as it should be.

Modeling AND Gate

$$y = \begin{cases} 0 & \text{if } I < 0.8 \\ 1 & \text{otherwise} \end{cases}$$

The value of β is a heuristic. A low value slows down the weight changes, thus slowing down the training, but a high value can result in overcorrection, thereby causing the weights to oscillate in successive changes. For noisy training patterns—those in which there are minor inaccuracies in recording the attribute values, for instance, 0.001 being recorded instead of 0—a low value of β is usually recommended. Let us choose $\beta = 0.2$ for training the neuron to model an AND gate.

$$\Delta w(j) \leftarrow \beta(z - y)x(j)$$

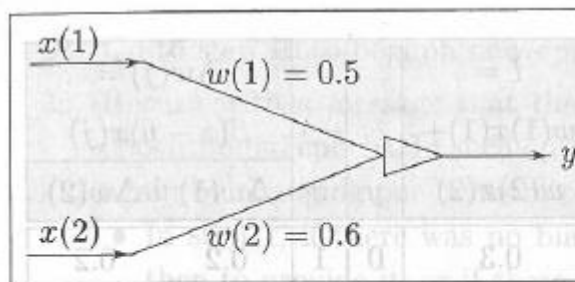
$$w(j) \leftarrow w(j) + \Delta w(j)$$

Modeling AND Gate

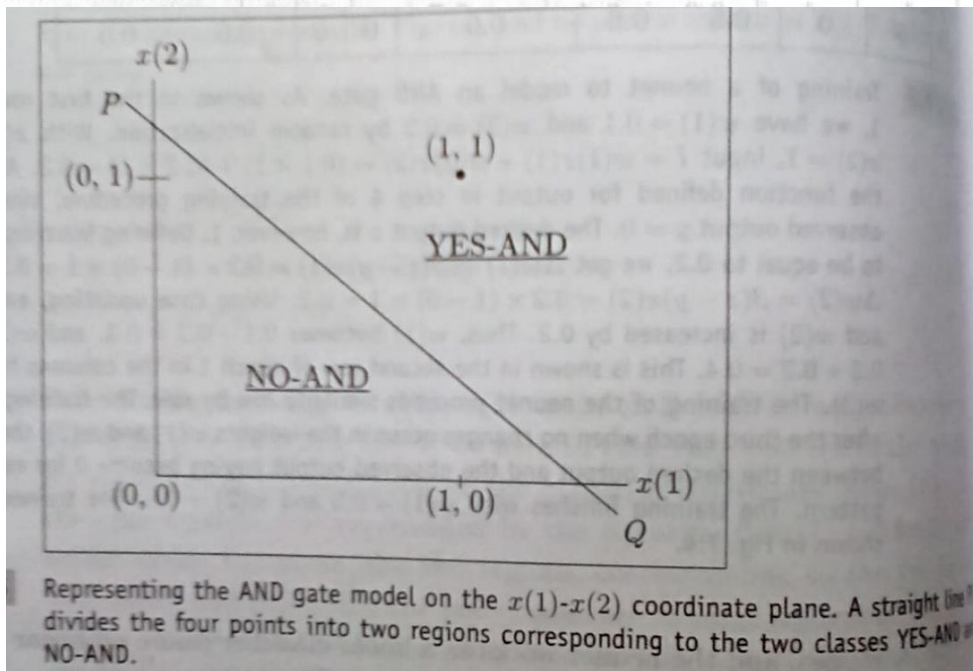
Epoch	$x(1)$	$x(2)$	Weights		$I = w(1)x(1) + w(2)x(2)$	y	z	$\Delta w(j) = \beta(z - y)x(j)$	
			$w(1)$	$w(2)$				$\Delta w(1)$	$\Delta w(2)$
1	1	1	0.1	0.2	0.3	0	1	0.2	0.2
	0	1	0.3	0.4	0.4	0	0	0.0	0.0
	1	0	0.3	0.4	0.3	0	0	0.0	0.0
	0	0	0.3	0.4	0.0	0	0	0.0	0.0
2	1	1	0.3	0.4	0.7	0	1	0.2	0.2
	0	1	0.5	0.6	0.6	0	0	0.0	0.0
	1	0	0.5	0.6	0.5	0	0	0.0	0.0
	0	0	0.5	0.6	0.0	0	0	0.0	0.0
3	1	1	0.5	0.6	1.1	1	1	0.0	0.0
	0	1	0.5	0.6	0.6	0	0	0.0	0.0
	1	0	0.5	0.6	0.5	0	0	0.0	0.0
	0	0	0.5	0.6	0.0	0	0	0.0	0.0

FIGURE 7.3 Training of a neuron to model an AND gate. As shown in the first row of epoch 1, we have $w(1) = 0.1$ and $w(2) = 0.2$ by random initialization. With $x(1) = 1$ and $x(2) = 1$, input $I = w(1)x(1) + w(2)x(2) = (0.1 \times 1) + (0.2 \times 1) = 0.3$. According to the function defined for output in step 4 of the training procedure, since $I < 0.8$, observed output $y = 0$. The desired output z is, however, 1. Defining learning constant β to be equal to 0.2, we get $\Delta w(1) = \beta(z - y)x(1) = 0.2 \times (1 - 0) \times 1 = 0.2$. Similarly, $\Delta w(2) = \beta(z - y)x(2) = 0.2 \times (1 - 0) \times 1 = 0.2$. Using case updating, each of $w(1)$ and $w(2)$ is increased by 0.2. Thus, $w(1)$ becomes $0.1 + 0.2 = 0.3$, and $w(2)$ becomes $0.2 + 0.2 = 0.4$. This is shown in the second row of epoch 1 in the columns for $w(1)$ and $w(2)$. The training of the neuron proceeds similarly row by row. The training is finished after the third epoch when no changes occur in the weights $w(1)$ and $w(2)$, the difference between the desired output and the observed output having become 0 for each training pattern. The training finishes with $w(1) = 0.5$ and $w(2) = 0.6$. The trained neuron is shown in Fig. 7.4.

Linear Classification in the AND Gate



A neuron modelling an AND gate. Input I to the neurode is $w(1)x(1) + w(2)x(2)$, where each of $x(1)$ and $x(2)$ can be either 0 or 1. By definition, output y is 0 if $I < 0.8$; otherwise, y is 1. With the weights as shown, y becomes 1 if both $x(1)$ and $x(2)$ are 1; otherwise, y becomes 0. You may want to confirm this by reading through the last four rows of the table in Fig. 7.3.



Analytical Arguments

We can find out the constraints on the weights in terms of the threshold t used in the output function, according to which

$$y = \begin{cases} 0 & \text{if } I < t \\ 1 & \text{otherwise} \end{cases}$$

For the first pattern in the training set given in Fig. 7.2, output y should be 1; for that to happen, input I should be greater than or equal to t , that is,

$$[w(1)x(1) + w(2)x(2)] \geq t \quad (7.5)$$

since $I = w(1)x(1) + w(2)x(2)$. From the second to the fourth patterns of Fig. 7.2, output y should be 0, for which, input I should be less than t , that is,

$$[w(1)x(1) + w(2)x(2)] < t \quad (7.6)$$

Substituting the values of $x(1)$ and $x(2)$ in Eqns (7.5) and (7.6), we obtain the following four equations:

$$[w(1) \times 1 + w(2) \times 1] \geq t \text{ (from the first pattern)} \quad (7.7)$$

$$[w(1) \times 0 + w(2) \times 1] < t \text{ (from the second pattern)} \quad (7.8)$$

$$[w(1) \times 1 + w(2) \times 0] < t \text{ (from the third pattern)} \quad (7.9)$$

$$[w(1) \times 0 + w(2) \times 0] < t \text{ (from the fourth pattern)} \quad (7.10)$$

Simplifying Eqns (7.7) to (7.9), we get the following constraints on the weights $w(1)$ and $w(2)$ for a neuron to model an AND gate

$$[w(1) + w(2)] \geq t \text{ [from Eqn (7.7)]}$$

$$w(2) < t \text{ [from Eqn (7.8)]}$$

$$w(1) < t \text{ [from Eqn (7.9)]}$$

Figure 7.3 shows that, with $t = 0.8$, the neuron finished its training at $w(1) = 0.5$ and $w(2) = 0.6$. Clearly, the weights satisfy the above constraints. As mentioned earlier, with different initial weights and a different value of the learning constant β , the neuron could have finished its training at, say, $w(1) = 0.3$ and $w(2) = 0.7$. These weights, too, would satisfy the above constraints. In general, the weight values at which a neuron finishes training need not be unique.

Modeling an OR Gate is similar to AND Gate;
However, story is different for an XOR Gate

Training pattern	Attributes corresponding to		Class	Desired output z of neunet
	$x(0,1)$	$x(0,2)$		
1	1	1	NO-XOR	0
2	0	1	YES-XOR	1
3	1	0	YES-XOR	1
4	0	0	NO-XOR	0

9 Training set used for a neunet to model an XOR gate.

Modeling XOR Gate is Not Possible for a Single Neural Node

Suppose, for some value of t , we define output y of the neurode as

$$y = \begin{cases} 0 & \text{if } I < t \\ 1 & \text{otherwise} \end{cases}$$

For the first and fourth patterns of the training set given in Fig. 7.9, output y should be 0, hence input I should be less than t , that is,

$$[w(1)x(0,1) + w(2)x(0,2)] < t \quad (7.21)$$

since $I = w(1)x(0,1) + w(2)x(0,2)$. For the second and third patterns of Fig. 7.9, output y should be 1; therefore input I should be greater than or equal to t , that is,

$$[w(1)x(0,1) + w(2)x(0,2)] \geq t \quad (7.22)$$

Substituting the values of $x(0,1)$ and $x(0,2)$ in Eqns (7.21) and (7.22), we obtain the following four equations:

$$[w(1) \times 1 + w(2) \times 1] < t \quad (\text{from the first pattern}) \quad (7.23)$$

$$[w(1) \times 0 + w(2) \times 1] \geq t \quad (\text{from the second pattern}) \quad (7.24)$$

$$[w(1) \times 1 + w(2) \times 0] \geq t \quad (\text{from the third pattern}) \quad (7.25)$$

$$[w(1) \times 0 + w(2) \times 0] < t \quad (\text{from the fourth pattern}) \quad (7.26)$$

Simplifying Eqns (7.23) to (7.25), we get the following constraints on the weights $w(1)$ and $w(2)$ for a neuron to model an XOR gate

$$[w(1) + w(2)] < t \quad [\text{from Eqn (7.23)}]$$

$$w(2) \geq t \quad [\text{from Eqn (7.24)}]$$

$$w(1) \geq t \quad [\text{from Eqn (7.25)}] \quad (7.17)$$

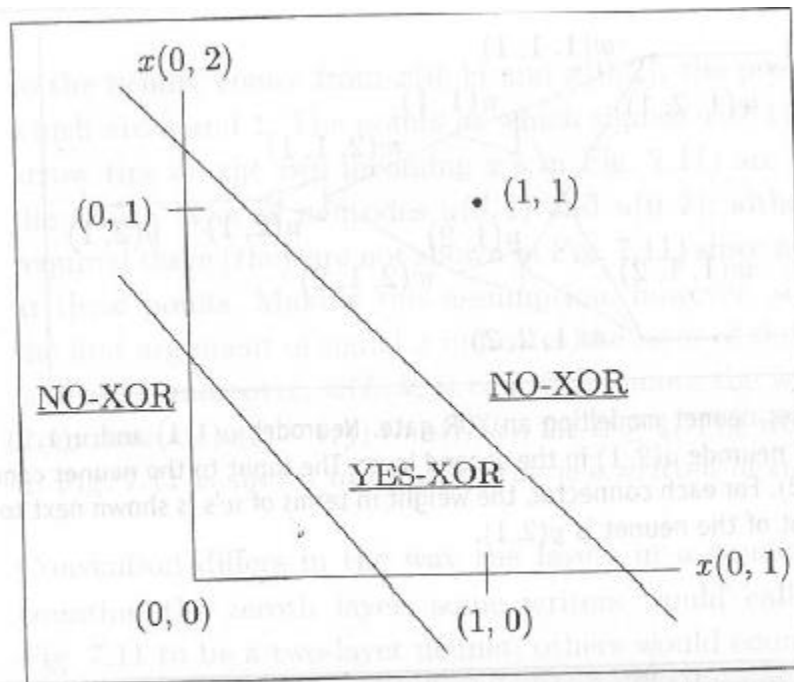
Adding Eqns (7.24) and (7.25), we get

$$[w(1) + w(2)] \geq 2t \quad (7.18)$$

But this contradicts Eqn (7.23), which says $[w(1) + w(2)] < t$. Therefore, an XOR gate cannot be modelled by a one-neurode neuron.

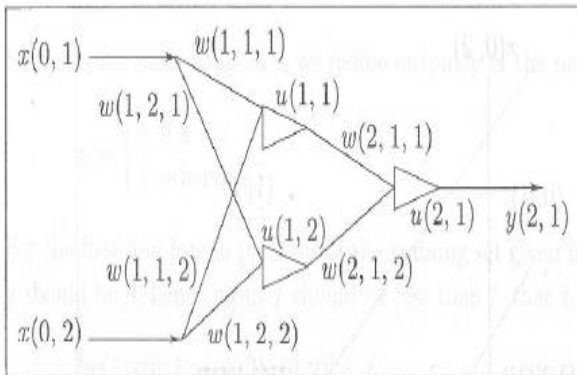
Figure 7.10 illustrates that, on the $x(0,1)$ - $x(0,2)$ coordinate plane, there does not exist a straight line that would divide the plane into two regions

XOR cannot be linearly separated by a straight line



- 10** On the $x(0, 1)$ - $x(0, 2)$ coordinate plane, there does not exist a straight line to separate the YES-XOR class from the NO-XOR class. Points $(0, 1)$ and $(1, 0)$ belong to the YES-XOR class, and points $(0, 0)$ and $(1, 1)$ belong to the NO-XOR class.

XOR Gate with 2-layer Network



A two-layer neuron modelling an XOR gate. Neurodes $u(1,1)$ and $u(1,2)$ are in the first layer, and neurode $u(2,1)$ in the second layer. The input to the neuron comes from $x(0,1)$ and $x(0,2)$. For each connector, the weight in terms of w 's is shown next to the connector. The output of the neuron is $y(2,1)$.

to the neuron comes from $x(0,1)$ and $x(0,2)$, the possible values of each of which are 0 and 1. The points at which signals $x(0,1)$ and $x(0,2)$ split (the arrow tips on the two incoming x 's in Fig. 7.11) are considered here to be the zeroth layer of neurodes $u(0,1)$ and $u(0,2)$, although no neurodes are required there (they are not shown in Fig. 7.11) since no computation is done at these points. Making this assumption, however, simplifies the notation: the first argument of signal x indicates the layer of the neurode into which x is flowing; moreover, $w(L,k,j)$ can then denote the weight of the connector from neurode $u(L-1,j)$ to $u(L,k)$, for $L \geq 1$. The weight of each connector in Fig. 7.11 is shown in terms of the w 's written next to the connector.

$$\begin{bmatrix} I(1,1) \\ I(1,2) \end{bmatrix} = \begin{bmatrix} w(1,1,1) = 1 & w(1,1,2) = -1 \\ w(1,2,1) = -1 & w(1,2,2) = 1 \end{bmatrix} \begin{bmatrix} x(1,1) \\ x(1,2) \end{bmatrix}$$

$$y(1,k) = \begin{cases} 0 & \text{if } I(1,k) < 0.1 \\ 1 & \text{otherwise} \end{cases}$$

Training Pattern	$x(0,1)$ that is $x(1,1)$	$x(0,2)$ that is $x(1,2)$	$I(1,1)$	$I(1,2)$	$y(1,1)$ that is $x(2,1)$	$y(1,2)$ that is $x(2,2)$	$I(2,1)$	$y(2,1)$
1	1	1	0	0	0	0	0	0
2	0	1	-1	1	0	1	1	1
3	1	0	1	-1	1	0	1	1
4	0	0	0	0	0	0	0	0

FIGURE 7.12 Outputs of the neuron shown in Fig. 7.11 modelling an XOR gate, for the four possible permutations of $x(0,1)$ and $x(0,2)$. Before entering the first-layer neurodes, signal $x(0,1)$ is renamed $x(1,1)$, and $x(0,2)$ is renamed $x(1,2)$. In the first row, for instance, given $x(1,1) = 1$ and $x(1,2) = 1$, we employ Eqn (7.27) by which $I(1,1) = w(1,1,1) \times x(1,1) + w(1,1,2) \times x(1,2)$, where $w(1,1,1) = 1$ and $w(1,1,2) = -1$, to get $I(1,1) = 0$. We again employ Eqn (7.27) by which $I(1,2) = w(1,2,1) \times x(1,1) + w(1,2,2) \times x(1,2)$, where $w(1,2,1) = -1$ and $w(1,2,2) = 1$, to get $I(1,2) = 0$. According to Eqn (7.28), since $I(1,1) < 0.1$, output from $u(1,1)$ is $y(1,1) = 0$; according to the same equation, since $I(1,2) < 0.1$, output from $u(1,2)$ is $y(1,2) = 0$. Outputs $y(1,1)$ and $y(1,2)$ are, respectively, renamed as inputs $x(2,1)$ and $x(2,2)$ to neurode $u(2,1)$. According to Eqn (7.29), we have $I(2,1) = w(2,1,1) \times x(2,1) + w(2,1,2) \times x(2,2)$, where $w(2,1,1) = 1$ and $w(2,1,2) = 1$; hence we get $I(2,1) = 0$. According to Eqn (7.30), since $I(2,1) < 0.1$, output from $u(2,1)$ is $y(2,1) = 0$, which is the desired output from the neuron, as given in the first row of Fig. 7.9. Similar calculations can be made for the other rows in this figure.

$$[I(2,1)] = [w(2,1,1) = 1 \quad w(2,1,2) = 1] \begin{bmatrix} x(2,1) \\ x(2,2) \end{bmatrix}$$

The output from neurode $u(2,1)$ is defined to be

$$y(2,1) = \begin{cases} 0 & \text{if } I(2,1) < 0.1 \\ 1 & \text{otherwise} \end{cases}$$

Conventions and Arguments for Having Different Layers

Convention differs in the way the layers in a neunet are counted: by not counting the zeroth layer, some writers would call the neunet given in Fig. 7.11 to be a two-layer neunet; others would count the zeroth layer and call the neunet to be a three-layer neunet. The former practice is preferred in this book—the neurodes in the zeroth layer are notional; they are imagined to be present only for notational simplicity. Accordingly, each of the neunets shown in Figs 7.1, 7.4, and 7.7 has one layer and the neunet given in Fig. 7.11 has two layers.

A neunet with a single layer can have more than one neurode in that layer, the output of no neurode flowing into another neurode in the same layer. According to the established result mentioned earlier in this section, classes that are not linearly separable cannot be recognized by a neunet containing only one neurode. In fact, the result is more general: classes which are not linearly separable cannot be recognized by a neunet containing only one layer of neurodes. For such classes—they are the ones often found in practice—we need neunets of more than one layer, as we do for the XOR-gate neunet shown in Fig. 7.11, in which signal $x(0,1)$ flows into $u(0,1)$, and $x(0,2)$ into $u(0,2)$. Since no computation is done at these two u 's, the output of each u is equal to its input: thus, the outputs from $u(0,1)$ and $u(0,2)$ are, respectively, $x(0,1)$ and $x(0,2)$.

Commonly used Architecture of Neural Net

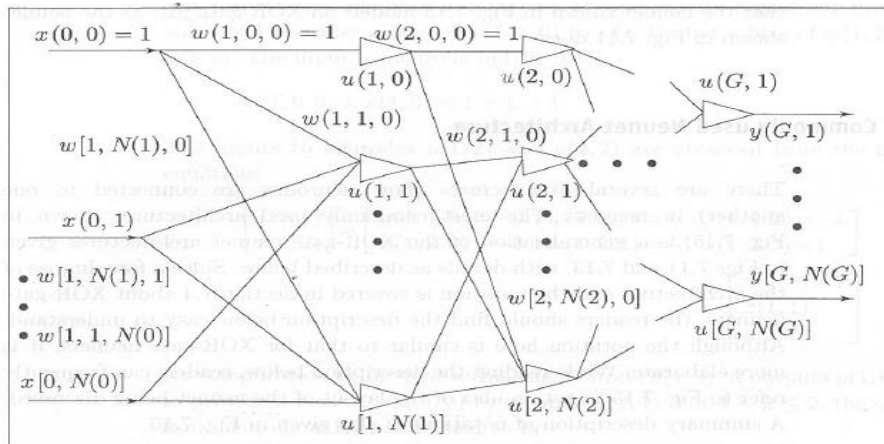


FIGURE 7.15 The most commonly used architecture of a neuenet. As explained in Section 7.5, the neuenet is feedforward, strictly layered, and fully connected. The u 's are neurodes, the x 's are signals flowing through the connectors, the w 's are the weights of the connectors, and the y 's are the outputs.

The neuenet contains $G \geq 1$ layers, which are numbered $0, 1, 2, \dots, G$. Every layer $0 \leq L \leq G$ contains neurodes $u(L, 1), u(L, 2), \dots, u[L, N(L)]$, where $N(L)$ is an element of array N , such that each element of the array has a value of 1 or more. Layers $L = 0, 1, \dots, (G - 1)$ may, in addition, each contain a neurode $u(L, 0)$ called the *biasing neurode*. The layer G does not contain any biasing neurode. Neurodes that are not biasing are called *non-biasing neurodes*. Since $N(L) \geq 1$, each layer has at least one non-biasing neurode. It will be assumed for this description that biasing neurodes are present. In case of neuennets that do not have any biasing neurodes, ignore whatever is said about biasing neurodes. In layer $0 \leq L < G$, if a variable ranges over 0 to $N(L)$ with biasing neurodes, then it ranges over 1 to $N(L)$ without biasing neurodes. As we did in Section 7.2, let us have variable $B = 0$ if biasing neurodes are present, and $B = 1$ otherwise. The neuenet shown in Fig. 7.15 depicts a *feedforward*, *strictly layered*, and *completely connected* neuenet.

Feedforward: Signals flow from neurodes in layer L to neurodes in layers greater than L , but not to neurodes in L itself, or to neurodes in layers less than L , for $0 \leq L < G$. Signal from a neurode in layer G does not flow to any other neurode. It is considered to be an output of the neuenet.

Strictly layered Signals flow from layer L only to layer $(L + 1)$, for $0 \leq L < G$. Thus, for instance, although the neuenet is feedforward, signals do not flow from layer L to, say, layer $(L + 2)$.

Completely connected For $0 \leq L < G$, (a) every non-biasing neurode in layer L is connected to send signals to every non-biasing neurode in layer $(L + 1)$, a non-biasing neurode in layer L thus not sending a signal to the biasing neurode in layer $(L + 1)$; and (b) the biasing neurode in layer L is connected to send signals to every neurode in layer $(L + 1)$, the biasing neurode in layer L thus providing a bias to the neurodes in layer $(L + 1)$.

Common Notations

- G output layer in the neunet with layers numbered $0, 1, \dots, G$.
- B indicates whether the neunet has biasing neurodes ($B = 0$) or does not have biasing neurodes ($B = 1$).
- $N(L)$ number of non-biasing neurodes in layer $0 \leq L \leq G$.
- $u(L, j)$ neurode j in layer $0 \leq L < G$, for $B \leq j \leq N(L)$. When $j = 0$, then $u(L, j)$ is a biasing neurode, otherwise a non-biasing neurode. The output layer has no biasing neurode; hence when $L = G$, we have $1 \leq j \leq N(G)$.
- $w(L, k, j)$ weight of the connector from neurode $u(L-1, j)$ to neurode $u(L, k)$, for layer $1 \leq L \leq G$, where $1 \leq k \leq N(L)$ and $B \leq j \leq N(L-1)$. Furthermore, $w(L, 0, 0) = 1$ is the weight of the connector from biasing neurode $u(L-1, 0)$ to biasing neurode $u(L, 0)$, for layer $1 \leq L < G$.
- $x(L, j)$ signal that flows from neurode $u(L-1, j)$ to the neurodes of layer $1 \leq L \leq G$, where $B \leq j \leq N(L-1)$, with the constraint that no signal flows from a non-biasing neurode in layer $L-1$ to a biasing neurode in layer L , since no connector exists between these neurodes. Moreover, $x(0, 0), x(0, 1), \dots, x(0, N(0))$ flow into the zeroth layer. By convention, $x(L, 0) = x(L-1, 0) = \dots = x(0, 0) = 1$ since these signals provide the bias.
- $I(L, k)$ input to neurode $u(L, k)$, for $1 \leq L \leq G$ and $B \leq k \leq N(L)$.
- $a(L, j)$ activation value of neurode $u(L, j)$, for $0 \leq L \leq G$, where $B \leq j \leq N(L)$.
- $y(G, k)$ observed output of neurode $u(G, k)$, for $1 \leq k \leq N(G)$.
- $z(G, k)$ desired output of neurode $u(G, k)$, for $1 \leq k \leq N(G)$.

Activation Functions: Sigmoid

Sigmoid, or logistic, function

$$\mathcal{A}(I) = \frac{1}{1 + e^{-cI}}$$

Often c is chosen to be 1. At $I = 0$, we get $\mathcal{A}(I) = 0.5$. Moreover,

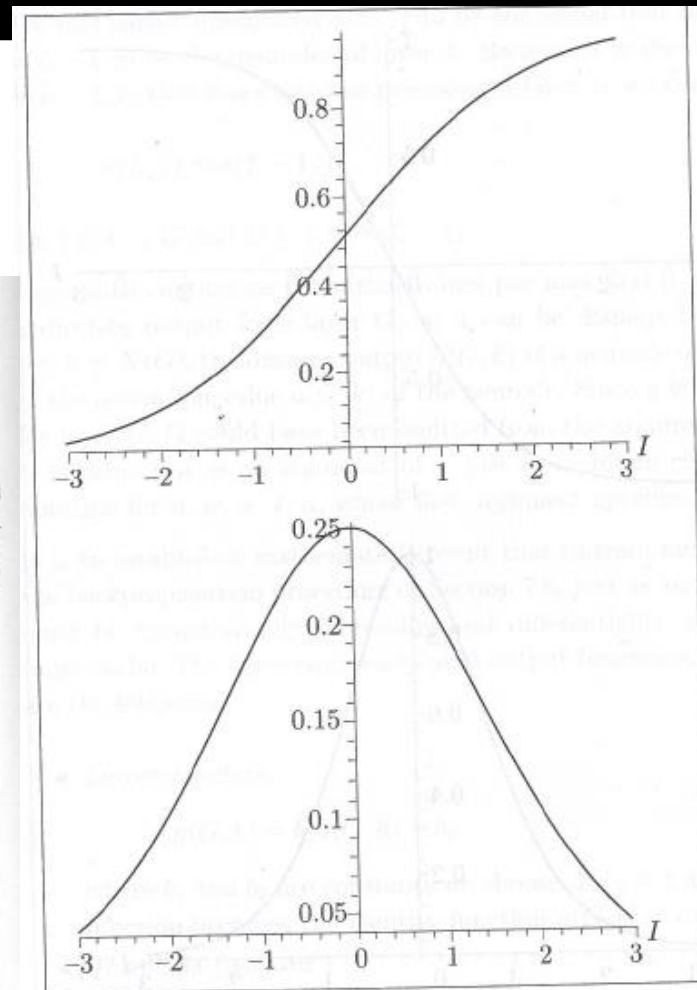
$$\lim_{I \rightarrow -\infty} \mathcal{A}(I) \rightarrow 0$$

$$\lim_{I \rightarrow \infty} \mathcal{A}(I) \rightarrow 1$$

Thus, $\mathcal{A}(I)$ lies between 0 and 1, regardless of the value of I . This prevents the activation value from ever becoming so large as to overflow the capacity of computation of the neuron. The derivative $\mathcal{A}'$ of $\mathcal{A}(I)$ with respect to I is

$$\begin{aligned}\mathcal{A}'(I) &= \frac{ce^{-cI}}{(1 + e^{-cI})^2} \\ &= \frac{c(1 + e^{-cI} - 1)}{(1 + e^{-cI})^2} \\ &= \frac{c}{1 + e^{-cI}} \left(1 - \frac{1}{1 + e^{-cI}}\right) \\ &= c\mathcal{A}(I)[1 - \mathcal{A}(I)] \text{ [from Eqn (7.39)]}\end{aligned}$$

For $c = 1$, the plots of $\mathcal{A}(I)$ and its derivative are shown in Fig. 7.18.



8 Curves obtained by plotting the sigmoid function $1/(1 + e^{-I})$ at the top and its derivative $e^{-I}/(1 + e^{-I})^2$ below.

Activation Functions: Hyperbolic Tangent

Hyperbolic function $\tanh(I)$

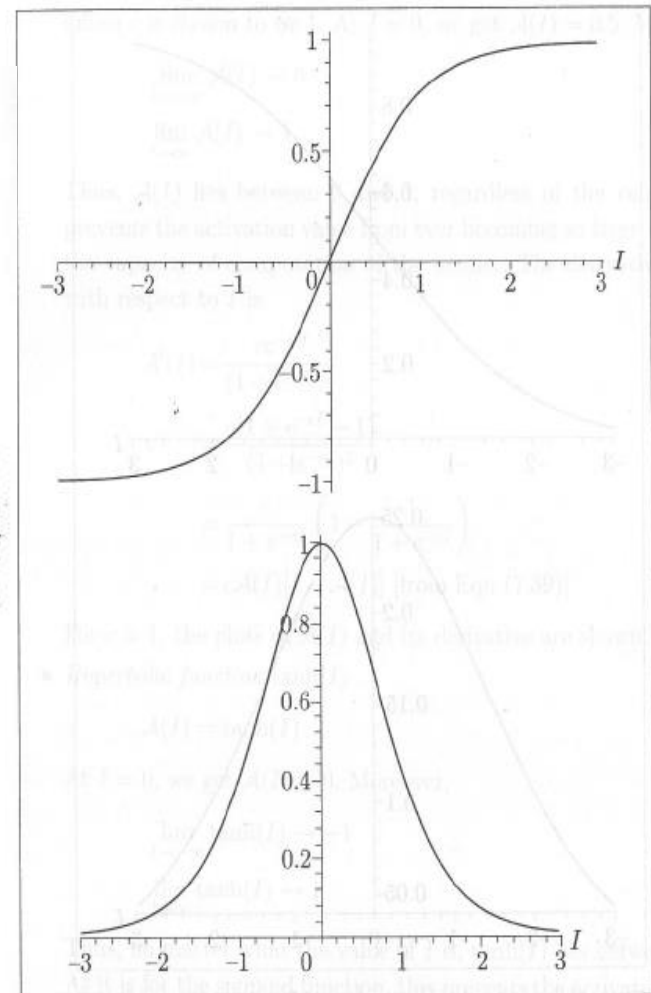
$$\mathcal{A}(I) = \tanh(I)$$

At $I = 0$, we get $\mathcal{A}(I) = 0$. Moreover,

$$\lim_{I \rightarrow -\infty} \tanh(I) \rightarrow -1$$

$$\lim_{I \rightarrow \infty} \tanh(I) \rightarrow 1$$

Thus, no matter what the value of I is, $\tanh(I)$ lies between -1 and 1 . As it is for the sigmoid function, this prevents the activation value from ever becoming so large as to overflow the capacity of computation of the neuron. The plots of $\tanh(I)$ and its derivative $\text{sech}^2(I)$ are shown in Fig. 7.19.



Curves obtained by plotting the hyperbolic function $\tanh(I)$ at the top, and its derivative $\text{sech}^2(I)$ below.

Output Functions

It is an established mathematical result that to train multilayer neurons by the backpropagation procedure of Section 7.6, just as an activation function must be monotonically increasing and differentiable, so must an output function be. The three commonly used output functions, for $1 \leq k \leq N(G)$, are the following:

- *Linear function*

$$y(G, k) = b_1 a(G, k) + b_2$$

where b_1 and b_2 are constants we choose. If $b_1 = 1$ and $b_2 = 0$, then the function becomes the identity function $y(G, k) = a(G, k)$.

- *Threshold function*

$$y(G, k) = \begin{cases} 0 & \text{if } a(G, k) < t \\ v & \text{otherwise} \end{cases}$$

where t is a threshold value, and v is some value. So $y(G, k)$ has a value of 0 or v . We choose the values for t and v , although typically $t < 1$ and $v = 1$.

- *Boundary function*

$$y(G, k) = \begin{cases} 0 & \text{if } a(G, k) < t_1 \\ a(G, k) & \text{if } t_1 \leq a(G, k) \leq t_2 \\ 1 & \text{otherwise} \end{cases}$$

Backpropagation Algorithm

The backpropagation procedure to train a multilayer neunet is an extension of the procedure to train a one-neurode neunet described in Section 7.2. We feed training patterns one by one to the neunet, and then we update the weights depending on the error. The backpropagation procedure is said to perform a 'gradient descent' on the error surface in the weight space: the weights change in the direction of steepest descent on the error surface. In other words, the weights change in the direction of the greatest rate of decrease of error.

Backpropagation Algorithm:

Change in weight to reduce error

In a multilayer neunet, for a given training pattern, error $e(G, k)$ in the output of neurode $u(G, k)$, for $1 \leq k \leq N(G)$, is defined to be

$$e(G, k) = \mathcal{A}'[I(G, k)] \times [z(G, k) - y(G, k)] \quad (7.40)$$

in which $[z(G, k) - y(G, k)]$ is the difference between the desired output $z(G, k)$ and the observed output $y(G, k)$. The derivative $\mathcal{A}'[I(G, k)]$ gives the rate of change of the activation function $\mathcal{A}$ at $I(G, k)$, the value input to neurode $u(G, k)$. The derivative scales the difference $[z(G, k) - y(G, k)]$.

Backpropagation Algorithm:

Change in weight to reduce error

Since the layers $0 \leq L < G$ are not output layers, the desired output of neurodes in these layers is not known. Error $e(L, j)$ in the output of neurode $u(L, j)$, where $B \leq j \leq N(L)$, in these layers is sent to all neurodes in layer $(L + 1)$. Thus, $e(L, j)$ contributes to error $e(L + 1, k)$ of neurode $u(L + 1, k)$, for $1 \leq k \leq N(L + 1)$, that is, $e(L, j)$ contributes to the error of each of the neurodes in layer $(L + 1)$. Error $e(L, j)$ is distributed over the neurodes of layer $(L + 1)$ in proportion to the weight of the connector between $u(L, j)$ and the different neurodes. The derivative $\mathcal{A}'[I(L, j)]$ —which gives the rate of change of the activation function $\mathcal{A}$ at $I(L, j)$, the value input to $u(L, j)$ —is used to scale the error; hence $e(L, j)$ is defined to be

$$e(L, j) = \mathcal{A}'[I(L, j)] \sum_{k=1}^{N(L+1)} w(L + 1, k, j) e(L + 1, k) \quad (7.41)$$

for $0 \leq L < G$ and $B \leq j \leq N(L)$. Thus, errors in the neurodes of layer L can be obtained from the errors in the neurodes of layer $(L + 1)$.

Backpropagation Algorithm:

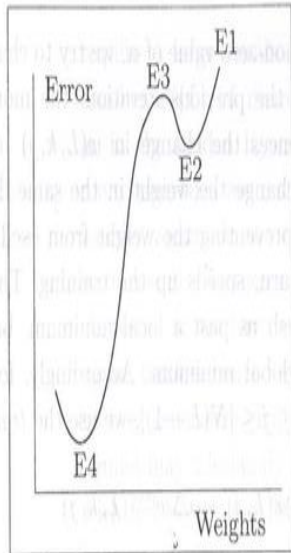
Change in weight to reduce error

To train a multilayer neuron, we commence by evaluating errors in the output layer G , knowing which we can evaluate the errors in layer $(G - 1)$, which in turn we use to evaluate the errors in layer $(G - 2)$, and so on, till we evaluate the errors in layer 1. As an outcome of this, we iteratively update weights starting from output layer G , then layer $(G - 1)$, then $(G - 2)$, and so on, till layer 1; that is, we first update all $w(G, k, j)$'s, then all $w(G - 1, k, j)$'s, and so on, till we update all $w(1, k, j)$'s. In other words, we first update the weights of the connectors between layers $(G - 1)$ and G , then between layers $(G - 2)$ and $(G - 1)$, and so on, till we update the weights of the connectors between layers 0 and 1.

Backpropagation Algorithm: Change in weight to reduce error

In step 20.1 of the training procedure for a one-neurode neunet (Section 7.2), the amount of change required in weight $w(j)$ depended on $\Delta w(j) = \beta(z - y)x(j)$, where β was the learning constant with a user-selected value between 0 and 1, the difference between the desired output z and the observed output y was the error, and $x(j)$ was the signal flowing through the connector whose weight was $w(j)$. Correspondingly, for training a multilayer neunet, we ought to have $\Delta w(L, k, j) = \beta e(L, k)x(L, j)$, for $1 \leq L \leq G$, where $1 \leq k \leq N(L)$ and $B \leq j \leq N(L - 1)$. This is, however, sometimes not enough.

Backpropagation Algorithm: Change in weight to reduce error



0 A plot of error on the weight plane. Point E2 is a local minimum and E4 is minimum.

looks like the curve in Fig. 7.20. When we begin training, let us presume we are at point E1. As we reduce the error during the training, we arrive at point E2. If we use $\beta e(L, k)x(L, j)$ to calculate $\Delta w(L, k, j)$, then we may stop our training at E2, which we should not. E2 is a *local minimum*: there is another point, namely E4, where the error is less than the error at E2. Point E4 is the *global minimum*: there is no point at which the error is less than that at E4. Ideally, the training should stop at E4. While training, if we arrive at E2 from E1, we need some momentum to carry us past E2 to E3, from where we can then descend to E4. An analogy is how momentum keeps a sled going downhill even though the sled may pass over short flat stretches or bumps till the sled stops at the bottom of the hill: E2–E3 are the metaphoric bump and E4 is the bottom of the hill.

This momentum in training a neunet is provided by adding $\alpha \Delta w^{\text{prev}}(L, k, j)$ to $\beta e(L, k)x(L, j)$, for calculating the value of $\Delta w(L, k, j)$. In $\alpha \Delta w^{\text{prev}}(L, k, j)$, known as the *momentum term*, $0 \leq \alpha \leq 1$ is the *momentum constant* (also known as the ‘inertia constant’), whose value we can select: and $\Delta w^{\text{prev}}(L, k, j)$ is the value of $\Delta w(L, k, j)$ from the previous iteration,

Backpropagation Algorithm: Change in weight to reduce error

iteration. This attempts to change the weight in the same direction as in the previous iteration, thus preventing the weight from oscillating around a local minimum. This, in turn, speeds up the training. The momentum term is strong enough to push us past a local minimum, but not strong enough to push us past a global minimum. Accordingly, for $1 \leq L \leq G$, where $1 \leq k \leq N(L)$, and $B \leq j \leq [N(L-1)]$, we use the *training formula*

$$\Delta w(L, k, j) = \beta e(L, k) x(L, j) + \alpha \Delta w^{\text{prev}}(L, k, j) \quad (7.42)$$

Backpropagation Algorithm: Steps

1. Fix the number G of layers to be used in the neunet. Researchers have argued that a two-layer neunet using the sigmoid activation function can solve any classification problem provided no limit is placed on the number of neurodes in the hidden layer. The neunet may accordingly need a large number of neurodes in the hidden layer. This does not mean that a two-layer neunet is the best. Neunets with more layers may be more efficient and effective but it is often recommended to begin with a few layers. If the neunet takes too long to train, or it does not recognize the training patterns well, then the number of layers can be increased to train the neunet.
2. Specify the number of non-biasing neurodes in each layer of the neunet. The stipulations of the classification problem for which the neunet is being designed will usually dictate the number of neurodes in layer zero and layer G : layer zero receives the attribute values of a given pattern, and layer G outputs values indicating the class of the pattern. A large number of attributes and how they are coded (an example on coding attributes is given in Section 7.7) for input could require a large number $N(0)$ of neurodes in layer zero. Similarly, a large number of classes and how they are coded for output could

Backpropagation: Steps

require a large number $N(G)$ of neurodes in layer G . Decide the number $N(L)$ of neurodes¹ to be put in layers $1 \leq L < G$, that is, in the hidden layers. It has been found in practice that if too many neurodes are put in the hidden layers, then the neunet will usually recognize the training patterns well, but not so the recall patterns: the neunet can be said to have memorized the training patterns, or to have learned the training patterns by rote. Then again, too few neurodes in the hidden layers usually cause the neunet to over-generalize. Training the neunet may take long, and still the neunet may misclassify many recall patterns. A suggestion is to begin with few layers and few neurodes. Train the neunet. If it takes too long to train, or if it misrecognizes too many patterns, then increase the layers and the number of neurodes in each layer. The ideal is to have the fewest possible layers and neurodes for the neunet to do the job you want it to do. As is apparent from Eqn (7.36) and (7.37), the more layers and the more neurodes the neunet contains, the more weights the neunet will need to learn.

Backpropagation: Steps

3. For each class, specify the desired output $z(G, k)$ for every neurode $u(G, k)$, for $1 \leq k \leq N(G)$. No two classes should have the same desired output.
4. Decide whether to use biasing neurodes in the neunet. If biasing neurodes are used, assign 0 to variable B ; otherwise, assign 1 to B . Equation 7.38 shows how many more weights the neunet has to learn with biasing neurodes than without biasing neurodes. The more weights the neunet has to learn, the slower the training will be. A suggestion is to begin without biasing neurodes. If the neunet misrecognizes too many patterns, then add biasing neurodes to the neunet, and try to train the neunet again.
5. Select the activation function $\mathcal{A}$ for neurode $u(L, k)$, where $1 \leq L \leq G$ and $1 \leq k \leq N(L)$. The function should be monotonically increasing and differentiable. The sigmoid or the tanh function is recommended. It has been observed that the sigmoid function is selected more often in practice.

Backpropagation: Steps

6. Define observed output $y(G, k)$ to be some function of the activation value $a(G, k)$ of neurode $u(G, k)$, for $1 \leq k \leq N(G)$.
7. Decide whether case updating or epoch updating of the weights should be used. As mentioned in Section 7.2, in case updating, the weights are updated after processing every training pattern, whereas, in epoch updating, the weights are updated after processing all the training patterns.
8. Fix a heuristic value of the *learning constant* $0 < \beta \leq 1$. As mentioned in Section 7.2, a low value of β slows down the training, but a high value of β can cause the weights to oscillate. If training is to be done only once—typically for classification problems that do not change over time—then slow training may be considered to be acceptable. Often, β takes a value of 0.5. If, however, the training patterns are noisy, then a lower value of β may be more appropriate.
9. Choose a heuristic value for the momentum constant $0 \leq \alpha \leq 1$. Often, in practice, $\alpha = 0.9$.
10. Specify a heuristic value for `epoch_bound` to indicate within how many epochs the training of the net is expected to finish, that is, how many times each training pattern is expected to be processed.

Backpropagation: Steps

11. Let $|V|$ be the number of training patterns in the training set V . It is desirable to have approximately equal number of patterns from each class; otherwise the neuronnet may learn frequent classes well, but the infrequent classes not so well. Moreover, it has been found in practice that it is better if the training patterns are fed (step 22) to the neuronnet in random order rather than all patterns of a class sequentially one after another. Since the patterns are fed to the neuronnet in the order in which they occur in the training set, the patterns should be placed in a random order in the training set.
12. If $B = 0$, then do
for $L = 1, 2, \dots, (G - 1)$ do
 $I(L, 0) \leftarrow 1$
 If biasing neurodes are present, then input to each of them is 1. This value will be needed in step 24 to calculate the error in layers 1 to $(G - 1)$.
13. For $L = 1, 2, \dots, G$ do
 for $k = 1, 2, \dots, N(L)$ do
 for $j = B, (B + 1), \dots, N(L - 1)$ do steps 13.1 and 13.2.

Backpropagation: Steps

13.1. Initialize $w(L, k, j)$ to a non-zero random real value.

These are the weights the neunet has to learn. The random values will typically be a mixture of positive and negative ones. Obtaining random values was discussed in Section 4.4. If all initial weights are identical, then it can happen that all weights change identically during the training. This can hamper the training of the neunet. So initial weights should not all be the same.

13.2. $\Delta w^{\text{prev}}(L, k, j) \leftarrow 0$

Initialized to zero for the first iteration, $\Delta w^{\text{prev}}(L, k, j)$ is used in step 25.1 to calculate $\Delta w(L, k, j)$.

14. epoch $\leftarrow 0$

Initialize variable epoch to zero so that it can count the epochs.

15. epoch $\leftarrow$ epoch + 1

Increment the counter for epochs by 1.

16. If epoch > epoch_bound, then go to step 30 to report failure in training the neunet.

17. For $L = 1, 2, \dots, G$ do

for $k = 1, 2, \dots, N(L)$ do

for $j = B, (B + 1), \dots, N(L - 1)$ do

$s(L, k, j) \leftarrow w(L, k, j)$

The weights at the beginning of the epoch are saved in array s . If it is found in step 27 that the neunet was already trained, then the procedure will return with the weights as they were at the beginning of the epoch.

Backpropagation: Steps

or the epoch.

18. If epoch updating was chosen in step 7, then

for $L = 1, 2, \dots, G$ do

for $k = 1, 2, \dots, N(L)$ do

for $j = B, (B + 1), \dots, N(L - 1)$ do

$q(L, k, j) \leftarrow 0$

Element $q(L, k, j)$ is initialized to zero here, so that, in steps 25.3 and 28, it can be used as an accumulator to sum up the amount by which $w(L, k, j)$ should be updated.

19. $T \leftarrow 0$

Initialize variable T to zero so that it can be used as a counter for the training patterns within each epoch.

20. $T \leftarrow T + 1$

Backpropagation: Steps

Increment the counter for the training patterns by 1.

21. If $T > |V|$, that is, all training patterns have been processed in the current epoch, then go to step 27 and check whether the neuronet had been trained by the end of the epoch.

22. Feed signals $x(0, 1)$ to $x[0, N(0)]$ to the neuronet, the signals representing the attribute values of the training pattern T . Invoke the forward propagation procedure given in Section 7.5 to obtain the values of $I(L, k)$, for $1 \leq L \leq G$, where $1 \leq k \leq N(L)$. Moreover, obtain the values of $y(G, 1), y(G, 2), \dots, y[G, N(G)]$.

23. For $k = 1, 2, \dots, N(G)$ do

$$e(G, k) \leftarrow \mathcal{A}'[I(G, k)] \times [z(G, k) - y(G, k)]$$

This is taken from Eqn (7.40), where $e(G, k)$ is the error in the output of neurode $u(G, k)$ in the output layer, and $\mathcal{A}'[I(G, k)]$ is the derivative of the activation function $\mathcal{A}$ at $I(G, k)$, the value input to $u(G, k)$. The term $[z(G, k) - y(G, k)]$ gives the difference between the desired output for the class of the pattern T and the observed output.

24. For $L = (G - 1), (G - 2), \dots, 1$ do

for $j = B, (B + 1), \dots, N(L)$ do

$$e(L, j) \leftarrow \mathcal{A}'[I(L, j)] \sum_{k=1}^{N(L+1)} w(L + 1, k, j) e(L + 1, k)$$

This is taken from Eqn (7.41), where $e(L, j)$ is the error in the output of neurode $u(L, j)$ in a non-output layer. $\mathcal{A}'[I(L, j)]$ is the derivative of the activation function $\mathcal{A}$ at $I(L, j)$, the value input to $u(L, j)$.

Backpropagation: Steps

25. For $L = 1, 2, \dots, G$ do
for $k = 1, 2, \dots, N(L)$ do
for $j = B, (B + 1), \dots, N(L - 1)$ do steps 25.1 to 25.4.

25.1. $\Delta w(L, k, j) \leftarrow \beta e(L, k)x(L, j) + \alpha \Delta w^{\text{prev}}(L, k, j)$
This is taken from Eqn (7.42).

25.2. If case updating was chosen in step 7, then do
 $w(L, k, j) \leftarrow w(L, k, j) + \Delta w(L, k, j)$
Weight $w(L, k, j)$ is modified by $\Delta w(L, k, j)$.

25.3. If epoch updating was chosen in step 7, then do
 $q(L, k, j) \leftarrow q(L, k, j) + \Delta w(L, k, j)$

In the current epoch, the modifications required in $w(L, k, j)$ are accumulated in $q(L, k, j)$.

- 25.4. $\Delta w^{\text{prev}}(L, k, j) \leftarrow \Delta w(L, k, j)$
Update $\Delta w^{\text{prev}}(L, k, j)$ so it can be used the next time
 $\Delta w(L, k, j)$ is calculated.

Steps

26. Go to step 20 to feed the neunet with the current epoch's next training pattern.
27. For at least p per cent of the training patterns in the last epoch, if the differences between each observed output $y(G, k)$ and its corresponding desired output $z(G, k)$, for $1 \leq k \leq N(G)$, was less than or equal to some heuristically chosen small value ϵ , say 0.05, then consider the training to be finished. If we choose p to be less than 100, then we are allowing $(100 - p)$ per cent of the training patterns to be misclassified by the trained neunet. If training is not finished, go to step 28; otherwise, it means that the weights at the beginning of the epoch were the ones to be learned by the neunet. If epoch updating was chosen in step 7, then the weights have not yet changed in the current epoch, hence go to step 31 to return with the weights. If, however, we have done case updating in step 25.2, then the weights have changed during the epoch, and we have to restore the weights as they were at the beginning of the epoch. We had saved the weights in array s in step 17. Consequently,
- for $L = 1, 2, \dots, G$ do
 - for $k = 1, 2, \dots, N(L)$ do
 - for $j = B, (B + 1), \dots, N(L - 1)$ do
 - $w(L, k, j) \leftarrow s(L, k, j)$
- Go to step 31 to return with the weights.

28. If epoch updating was chosen in step 7, then
- for $L = 1, 2, \dots, G$ do
 - for $k = 1, 2, \dots, N(L)$ do
 - for $j = B, (B + 1), \dots, N(L - 1)$ do
 - $w(L, k, j) \leftarrow w(L, k, j) + q(L, k, j)$
- After the epoch, the weights are updated by the accumulated modification required for all of the patterns in the training set.
29. Go to step 15 to begin a new epoch.

Backpropagation: Steps

30. Return with a message that the neunet failed to be trained within epoch_bound epochs. If so, you can try to train the neunet again by changing, for instance, one or more of the following:

- The number G of layers in step 1
- The number of non-biasing neurodes in one or more layers in step 2
- In step 4, if there were no biasing neurodes, then insert them; or if there were biasing neurodes, then remove them
- The activation function selected in step 5
- The definition of the output function in step 6
- The method of updating the weights chosen in step 7, case updating to epoch updating, or conversely
- The value of the learning rate β selected in step 8
- The value of the momentum rate α selected in step 9
- The value of the epoch_bound chosen in step 10
- The initial random values of the weights $w(L, k, j)$ in step 13.1
- The value of p in step 27
- The value of ϵ in step 27

31. Return from the procedure with weights $w(L, k, j)$ for $1 \leq L \leq G$, where $1 \leq k \leq N(L)$ and $B \leq j \leq N(L - 1)$. These are the weights learned by the neunet.

Professor – Student Neural Net

Let us have $G = 2$, because it has been observed in practice that most classification problems can be solved by a two-layer neunet. The neunet thus has layers 0, 1, and 2.

The attributes are HABIT having two values (gabby and quiet), EATS having three values (baked, fried, and roasted), and FOOTWEAR having two values (clogs and sandals). We need to code these non-numeric values into numeric values, since the signals flowing through a neunet are numeric. Let us code these non-numeric values as shown in Fig. 7.21. The figure shows we shall have seven signals $x(0, 1)$ to $x(0, 7)$ flowing into the zeroth layer. Thus the input layer needs seven neurodes, that is, $N(0) = 7$.

Attribute	Value	Signals entering zeroth layer of neunet		
HABIT	= Gabby	$x(0, 1) = 0$	$x(0, 2) = 1$	
	= Quiet	$x(0, 1) = 1$	$x(0, 2) = 0$	
EATS	= Fried	$x(0, 3) = 0$	$x(0, 4) = 0$	$x(0, 5) = 1$
	= Roasted	$x(0, 3) = 0$	$x(0, 4) = 1$	$x(0, 5) = 0$
	= Baked	$x(0, 3) = 1$	$x(0, 4) = 0$	$x(0, 5) = 0$
FOOTWEAR	= Clogs	$x(0, 6) = 0$	$x(0, 7) = 1$	
	= Sandals	$x(0, 6) = 1$	$x(0, 7) = 0$	

FIGURE 7.21 Numeric codes chosen for the non-numeric attribute values for use by a neunet i.o be trained on the professor–student training patterns given in Fig. 1.1.

Professor – Student Neural Net

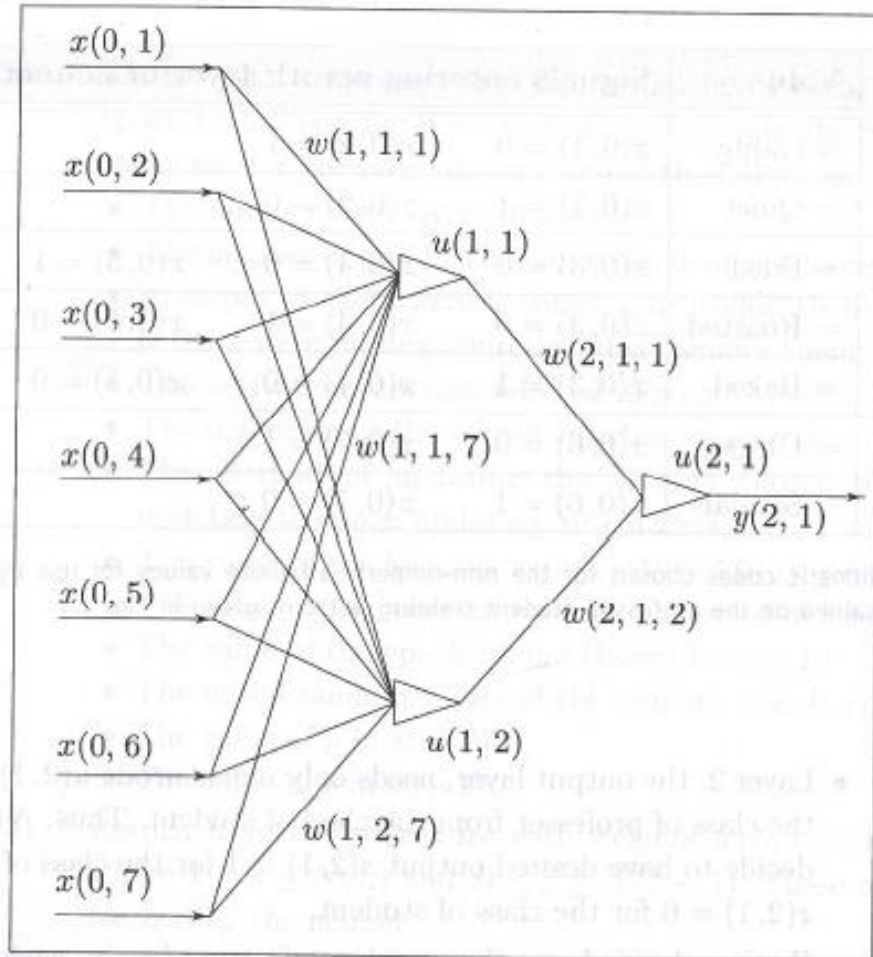
Layer 2, the output layer, needs only one neurode $u(2, 1)$ to distinguish the class of professor from the class of student. Thus, $N(2) = 1$. Let us decide to have desired output $z(2, 1) = 1$ for the class of professor, and $z(2, 1) = 0$ for the class of student.

Having decided on the number of neurodes in each of the input and output layers, we next need to decide on the number $N(1)$ of non-biasing neurodes in layer 1, the hidden layer. This number is a heuristic. Since too many neurodes in the hidden layer cause the neunet to memorize the training set, leading the neunet to recognize the training patterns well, but not so the recall patterns, let us choose a small value of $N(1)$, for instance, $N(1) = 2$.

Equation (7.38) shows that, if we have biasing neurodes in the neunet, it has to learn more weights, thereby slowing the training. So a usual recommendation is to start without biasing neurodes. If we fail to train the neunet, we can insert biasing neurodes and then try to train the neunet. Accordingly, let us choose not to have any biasing neurodes. With all the decisions made till now, we can say that the neunet will look like the one shown in Fig. 7.22.

The sigmoid function with constant $c = 1$, whose plot is shown in Fig. 7.18, is chosen to be the activation function. Thus, for $1 \leq L \leq 2$ and $1 \leq k \leq N(L)$, the activation value of neurode $u(L, k)$ with input

Professor – Student Neural Net



A net for recognizing the professor-student patterns given in Figs 1.1 and 1.2. The input layer has seven neurodes, the hidden layer has two neurodes, and the output layer has one neurode. No biasing neurodes are being used.

$I(L, k)$ is

$$a(L, k) = \frac{1}{1 + e^{-I(L, k)}}$$

- From neurode $u(2, 1)$, let us choose the observed output

$$y(2, 1) = \begin{cases} 0 & \text{if } a(2, 1) < 0.05 \\ a(2, 1) & \text{if } 0.05 \leq a(2, 1) \leq 0.95 \\ 1 & \text{otherwise} \end{cases}$$

Coupling $y(2, 1)$ with the desired output $z(2, 1)$ described above, we can say that, for a given pattern, when $y(2, 1) = 0$, then the pattern is being recognized as a student, and when $y(2, 1) = 1$, then the pattern is being recognized as a professor.

- Since epoch updating is less likely to cause oscillations in weights than case updating, let us opt for epoch updating to train the neunet.
- Let us choose learning constant $\beta = 0.5$ and momentum constant $\alpha = 0.9$.
- Let epoch_bound = 40, expecting that the training will finish within that many epochs.
- We will consider the training finished, when for 100% of the training patterns, the observed output $y(2,1)$ will be identical to the desired output $z(2,1)$.

With the choices made above, let us invoke a random number generator (discussed in Section 4.4) to initialize weights $w(L, k, j)$ for $1 \leq L \leq 2$, where $1 \leq k \leq N(L)$, and $1 \leq j \leq N(L-1)$. Let us suppose that the random initial values of the weights are as shown in the second column of Fig. 7.23. Then, if we are to implement the backpropagation training procedure of Section 7.6, we will notice that the training is finished in 11 epochs, and the weights become as shown in the third column of Fig. 7.23.

As an example, let us work through the forward propagation procedure of Section 7.5 to see how the signals would flow through the trained neunet for the training pattern T1 given in Fig. 1.1. It has these attribute values: HABIT equal to gabby, EATS equal to baked, and FOOTWEAR equal to clogs. According to Fig. 7.21, these values will be coded so that the following signals flow into the zeroth layer:

$$x(0, 1) = 0$$

$$x(0, 2) = 1$$

$$x(0, 3) = 1$$

$$x(0, 4) = 0$$

$$x(0, 5) = 0$$

$$x(0, 6) = 0$$

$$x(0, 7) = 1$$

Weight	Value after	
	Random initialization	Neunet's training
$w(1, 1, 1)$	-0.1352	-0.8368
$w(1, 1, 2)$	-0.1351	-0.9784
$w(1, 1, 3)$	-0.1949	-1.9110
$w(1, 1, 4)$	-0.0211	0.4040
$w(1, 1, 5)$	0.1647	-0.0889
$w(1, 1, 6)$	-0.0812	0.2308
$w(1, 1, 7)$	0.2428	-1.6140
$w(1, 2, 1)$	-0.1689	0.6690
$w(1, 2, 2)$	0.1917	0.0466
$w(1, 2, 3)$	0.1566	2.7410
$w(1, 2, 4)$	0.1102	-1.0610
$w(1, 2, 5)$	0.0469	-0.6733
$w(1, 2, 6)$	-0.1201	-1.6790
$w(1, 2, 7)$	0.1986	2.4510
$w(2, 1, 1)$	0.4979	18.3200
$w(2, 1, 2)$	0.1761	-7.0830

23 Weights of the connectors of the neunet given in Fig. 7.22 at the beginning of the training and after finishing the training.

Unchanged, the signals enter the first layer; hence $x(0, j)$ becomes $x(1, j)$, for $1 \leq j \leq 7$. Input to neurode $u(1, 1)$ is

$$I(1, 1) = \sum_{j=1}^7 w(1, 1, j) x(1, j) = -4.503$$

The values of the weights are taken from the third column of Fig. 7.23 and the results have been rounded off to three places after the decimal point.

Name of pattern	Signals $x(0, 1)$ to $x(0, 7)$	$I(1, 1)$	$I(1, 2)$	$a(1, 1)$	$a(1, 2)$	$I(2, 1)$	$a(2, 1)$	$y(2, 1)$
T1	01 100 01	-4.503	5.239	0.011	0.995	-6.845	0.001	0
T2	01 010 10	-0.344	-1.726	0.415	0.151	6.532	0.999	1
T3	01 100 10	-2.659	1.109	0.066	0.752	-4.126	0.016	0
T4	10 001 10	-0.695	-1.683	0.333	0.157	4.990	0.993	1
T5	01 001 01	-2.681	1.824	0.064	0.861	-4.925	0.007	0
T6	10 100 10	-1.797	1.731	0.142	0.850	-3.412	0.032	0
T7	01 001 10	-0.837	-2.306	0.302	0.091	4.896	0.993	1
T8	10 001 01	-2.540	2.447	0.071	0.920	-5.212	0.005	0
R1	10 100 01	-4.362	5.861	0.013	0.997	-6.833	0.001	0
R2	10 010 10	-0.202	-2.071	0.450	0.112	7.446	0.999	1
R3	01 010 01	-2.118	1.437	0.107	0.808	-3.757	0.023	0
R4	10 010 01	-2.047	2.059	0.114	0.887	-4.186	0.015	0

FIGURE 7.24 Values propagated when attribute values of the professor-student training patterns T1 to T8 given in Fig. 1.1 and the recall patterns R1 to R4 given in Fig. 1.2 are fed to the net given in Fig. 7.22. To understand the entries in the second column, consider, for instance, pattern T1, whose attribute values are HABIT = gabby, EATS = baked, and FOOTWEAR = clogs. According to the numeric codes chosen in Fig. 7.21, these attribute values are represented by the string 01 100 01, the space within the string being shown only for your ease in reading. This means that the signals fed to the zeroth layer of the net have the following values: $x(0, 1) = 0$, $x(0, 2) = 1$, $x(0, 3) = 1$, $x(0, 4) = 0$, $x(0, 5) = 0$, $x(0, 6) = 0$, and $x(0, 7) = 1$. If $y(2, 1) = 0$, then the corresponding pattern is recognized as a student, and if $y(2, 1) = 1$, then it is recognized as a professor. See Fig. 7.25.

Similarly, input to neurode $a(1, 2)$ is

$$I(1, 2) = \sum_{j=1}^7 w(1, 2, j) x(1, j) = 5.239$$

Professor – Student Neural Net

Then the activation value of neurode $u(1,1)$ is

$$a(1,1) = \frac{1}{1 + e^{-I(1,1)}} = 0.011$$

and the activation value of neurode $u(1,2)$ is

$$a(1,2) = \frac{1}{1 + e^{-I(1,2)}} = 0.995$$

Flowing to the second layer, $a(1,1)$ becomes $x(2,1)$, and $a(1,2)$ becomes $x(2,2)$. Then, input to neurode $u(2,1)$ is

$$I(2,1) = \sum_{j=1}^2 w(2,1,j) x(2,j) = -6.845$$

and the neurode's activation value is

$$a(2,1) = \frac{1}{1 + e^{-I(2,1)}} = 0.001$$

Since this activation value is less than 0.05, then according to the definition of output chosen above,

$$y(2,1) = 0$$

According to the desired output values defined above for the two classes, we can say that the neunet has recognized pattern T1 as a student.

We can carry out the above calculations for the remaining training patterns given in Fig. 1.1 and for the recall patterns given in Fig. 1.2. The values of the signals propagated through the trained neunet for all these patterns are given in Fig. 7.24. The classifications of these patterns are then given in Fig. 7.25, which shows that the trained neunet recognized all training patterns correctly. Moreover, the classifications of the recall patterns are incidentally identical to the Naive Bayes classifications shown in Fig. 5.2, identical to the nearest neighbour classifications shown in Fig. 6.1, and identical to the classifications shown in Fig. 2.11 for the decision tree shown in Fig. 2.9, the tree obtained by maximizing the ratio of information gain.

Professor – Student Neural Net

NAME of pattern	Attributes			Classification
	HABIT	EATS	FOOTWEAR	
T1	Gabby	Baked	Clogs	Student
T2	Gabby	Roasted	Sandals	Professor
T3	Gabby	Baked	Sandals	Student
T4	Quiet	Fried	Sandals	Professor
T5	Gabby	Fried	Clogs	Student
T6	Quiet	Baked	Sandals	Student
T7	Gabby	Fried	Sandals	Professor
T8	Quiet	Fried	Clogs	Student
R1	Quiet	Baked	Clogs	Student
R2	Quiet	Roasted	Sandals	Professor
R3	Gabby	Roasted	Clogs	Student
R4	Quiet	Roasted	Clogs	Student

Classifications of the professor–student training patterns T1 to T8 given in Fig. 1.1 and the recall patterns R1 to R4 given in Fig. 1.2 by the trained neuronnet shown in Fig. 7.22, with the weights as shown in the third column of Fig. 7.23, and the propagated values as given in Fig. 7.24.

Things to do

- Read Decision Tree Classifiers (Extensions) from Shinghal, 2011 (Chapter 3; Material posted on Moodle)
- Read Chapter 7 from Shinghal, 2011
- Read WEKA from Witten's book about decision trees
- Do Assignment 3



Questions or Comments?